

DAG 分解与实时调度仿真系统

基于 DAG（有向无环图）任务模型的实时调度分解与仿真系统，实现了论文中的 GEDF 及其变体（NP、DS、R、RDS）的可调度性分析与仿真验证。

项目结构

```
dag_task_system/
├── WORKSPACE                # Bazel 工作区定义
├── BUILD                    # Bazel 构建规则
├── .bazelrc                 # Bazel 构建配置
├── Makefile                 # 传统 Make 构建（保留）
├── include/
│   ├── dag_model.h         # DAG 数据模型：Vertex、DAGTask、Segment、SporadicTask
│   ├── decomposition.h     # DAG 分解算法接口
│   ├── gedf_simulator.h    # GEDF 仿真器与理论判定接口
│   ├── gedf_variants.h     # GEDF 变体（NP/DS/R/RDS）+ 开销 + 精度评估
│   ├── vertex_reassemble.h # 顶点重组（用于 GEDF-NP）
│   ├── dag_generators.h    # 随机 DAG 生成器（Erdős-Rényi  $G(n,p)$ ）
│   ├── stg_parser.h        # STG (Standard Task Graph) 格式解析器
│   ├── wfcommons_parser.h  # WfCommons/WfInstances JSON 格式解析器
│   └── json.hpp            # nlohmann/json 单头文件库
├── src/
│   ├── main.cpp            # 主程序入口（8 个实验 + STG/WfInstances 验证）
│   ├── decomposition.cpp   # 分解算法实现
│   ├── gedf_simulator.cpp  # GEDF 仿真实现
│   ├── gedf_variants.cpp   # GEDF 变体实现
│   ├── dag_generators.cpp  # DAG 生成器实现
│   ├── stg_parser.cpp      # STG 解析器实现
│   └── wfcommons_parser.cpp # WfInstances 解析器实现
├── tests/                  # 单元测试
│   ├── test_utils.h        # 测试工具
│   ├── dag_model_test.cpp
│   ├── decomposition_test.cpp
│   ├── gedf_simulator_test.cpp
│   ├── gedf_variants_test.cpp
│   ├── overhead_test.cpp
│   ├── precision_test.cpp
│   ├── dag_generators_test.cpp
│   └── sample_graphs.cpp
└── utils/
    └── gen_tool.cpp         # DAG 生成 CLI 工具
```

环境要求

- 编译器: GCC 7+ 或 Clang 10+（需支持 C++17）

- 构建工具: [Bazel](#) (推荐) 或 GNU Make
- 操作系统: Linux / macOS

构建方式

Bazel 构建 (推荐)

```
# 构建主程序
bazel build //:dag_simulator

# 优化构建
bazel build --config=opt //:dag_simulator

# 调试构建
bazel build --config=dbg //:dag_simulator

# 构建 DAG 生成工具
bazel build //:gen_tool

# 运行所有测试
bazel test //...

# 运行单个测试
bazel test //:dag_model_test
bazel test //:decomposition_test
```

Make 构建 (传统方式)

```
# 构建主程序
make all

# 构建测试
make tests

# 运行
make run

# 清理
make clean
```

使用方法

运行主程序

```
# 基本运行 (输出到 output/results.json)
bazel run //:dag_simulator
```

```
# 指定输出路径
bazel run //:dag_simulator -- -o /path/to/output.json

# 使用 STG 数据集
bazel run //:dag_simulator -- --stg /path/to/stg_directory

# 使用 WfInstances 数据集（目录或单文件）
bazel run //:dag_simulator -- --wf /path/to/wfinstances_directory
bazel run //:dag_simulator -- --wf /path/to/workflow.json

# 组合使用
bazel run //:dag_simulator -- -o results.json --stg ./stg_data --wf
./wf_data
```

命令行参数

参数	说明	示例
<code>-o <path></code>	指定 JSON 输出路径	<code>-o output/results.json</code>
<code>--stg <dir></code>	STG 数据集目录路径	<code>--stg data/stg/rnc50</code>
<code>--wf <path></code>	WfInstances 数据路径（目录或 JSON 文件）	<code>--wf data/wfinstances/montage</code>

DAG 生成工具

```
# 生成随机 DAG 并输出为 JSON
bazel run //:gen_tool -- output.json <n_vertices> <p> <wcet_min>
<wcet_max> <util_norm> <seed> [task_id]

# 示例：生成 30 个顶点、边概率 0.1、WCET 在 [10,50] 的 DAG
bazel run //:gen_tool -- dag.json 30 0.1 10 50 0.4 42
```

支持的数据格式

1. STG (Standard Task Graph Set)

来源： [早稻田大学笠原实验室](#)

```
# STG 文件格式（无通信开销版本）
# 第 1 行：N（顶点数，含 dummy 入口/出口）
# 后续行：task_id wcet num_preds pred1 pred2 ...
10
0 0 0
1 20 1 0
```

```
2 15 1 0
3 10 2 1 2
...
11 0 2 9 10
```

使用方式：将 `.stg` 文件放入目录，通过 `--stg` 参数指定。

2. WfCommons / WfInstances (JSON 格式)

来源：[WfCommons](#) / [WfInstances](#)

支持多种 JSON 格式变体：

WfFormat v1.x (推荐)：

```
{
  "name": "montage-workflow",
  "schemaVersion": "1.5",
  "workflow": {
    "execution": {
      "makespanInSeconds": 123.45,
      "tasks": [
        {
          "name": "mProjectPP_0",
          "type": "compute",
          "runtime": 12.34,
          "parents": [],
          "children": ["mDiffFit_0", "mDiffFit_1"]
        },
        {
          "name": "mDiffFit_0",
          "type": "compute",
          "runtime": 5.67,
          "parents": ["mProjectPP_0"],
          "children": ["mConcatFit_0"]
        }
      ]
    }
  }
}
```

旧版格式：

```
{
  "workflow": {
    "jobs": [
      {
        "name": "task_0",
        "runtime": 10.5,
```

```

    "parents": [],
    "children": ["task_1"]
  }
]
}
}

```

使用方式：

- 单文件：`--wf workflow.json`
- 目录（递归搜索 `.json` 文件）：`--wf /path/to/wfinstances/`

3. 随机生成（内置）

使用 Erdős-Rényi $G(n,p)$ 模型随机生成 DAG，无需外部数据文件。

实验模块

主程序包含以下实验（自动运行 Part 1-7，Part 8-9 需指定数据路径）：

编号	实验	说明
Part 1	Paper Example	论文示例 DAG (Fig.1) 的完整分解与仿真
Part 2	Random Task Set	随机任务集合的分解与调度
Part 3	Acceptance Ratio vs Util	验收率随归一化利用率变化
Part 4	Acceptance Ratio vs Processors	验收率随处理器数量变化
Part 5	Variants Comparison	GEDF / NP / DS 变体对比
Part 6	High Elasticity + Overhead	高弹性实验 + 开销敏感性分析
Part 7	Precision Evaluation	WCRT 精度评估
Part 8	STG Dataset	STG 数据集验证（需 <code>--stg</code> ）
Part 9	WfInstances Dataset	WfInstances 数据集验证（需 <code>--wf</code> ）

输出格式

所有实验结果以 JSON 格式输出，结构如下：

```

{
  "paper_example": { ... },
  "random_taskset": { ... },
  "acceptance_ratio_vs_util": [ ... ],
  "acceptance_ratio_vs_processors": [ ... ],
  "variants_comparison": [ ... ],
  "high_elasticity": [ ... ],
  "overhead_comparison": [ ... ],
}

```

```
"precision_evaluation": { ... },
"stg_experiment": { ... },
"wfinstances_experiment": { ... }
}
```

核心概念

- **DAGTask**: DAG 任务，包含顶点集合、周期 T、总执行时间 C、关键路径长度 L
- **Decomposition**: 将 DAG 分解为段 (Segment)，每段包含分配的顶点切片
- **SporadicTask**: 分解后得到的零散任务，继承父 DAG 顶点的片段
- **Ω (Omega)**: 结构特征值，衡量 DAG 的分解质量
- **GEDF**: Global Earliest Deadline First 调度算法
- **GEDF-NP**: 非抢占变体
- **GEDF-DS**: 密度分离变体
- **GEDF-R / GEDF-RDS**: 受限迁移变体

第三方依赖

- [nlohmann/json](#) v3.x — JSON 解析库 (已包含为 `include/json.hpp`)